Student Study & Marks Management System

VITyarthi – Build Your Own Project Report

Student Name: _____
Registration/Roll No.: _____
Course: _____

1. Introduction

This project is a Python console application designed to manage basic student academic records. It provides student management, marks management, result analysis and report generation in a modular and maintainable structure.

2. Problem Statement

Manual student record management and result calculation can lead to repeated work and calculation errors. The proposed system provides a lightweight local solution for maintaining records and calculating academic results.

3. Objectives

The objectives are to apply Python programming concepts to a real-world problem, implement modular functions, validate input, persist data, calculate results and generate reports.

4. Functional Requirements

The system shall add/list/search/delete students, add or update marks, calculate total and percentage, assign grades, determine pass/fail status, and generate a text report.

5. Non-functional Requirements

Usability, reliability, maintainability, error handling, performance for small datasets, and resource efficiency are addressed through a menu interface, JSON persistence, modular files, validation and exception handling.

6. System Architecture

The application uses a menu/controller layer in main.py, separate modules for students, marks, results and reports, a validation module, and a storage module backed by a JSON file.

7. Design Diagrams

Architecture, workflow, use-case, sequence and component/class diagrams are provided in docs/architecture.md and can be rendered or recreated as diagrams for the final submission.

8. Design Decisions & Rationale

Python modules separate responsibilities so that changes to marks, reports or storage can be made independently. JSON was selected because it is lightweight and easy to inspect for a small academic project.

9. Implementation Details

The implementation uses lists and dictionaries for data structures, functions for modularity, if/else conditions and loops for control flow, file handling for persistence, and validation/exception handling for invalid input.

10. Screenshots / Results

Add screenshots of the running menu, a student record, result output and generated report after running the project.

11. Testing Approach

Validation tests check grade calculation and input validation, including valid/invalid marks and valid/invalid student names. Additional manual tests should cover duplicate roll numbers, missing students and empty marks.

12. Challenges Faced

Typical challenges include maintaining consistent JSON structure, validating user input, separating functionality into modules and ensuring that result calculations handle missing marks.

13. Learnings & Key Takeaways

The project demonstrates practical use of Python functions, modules, dictionaries, lists, loops, conditions, file handling, validation, error handling and basic software design.

14. Future Enhancements

Possible improvements include a graphical interface, SQLite/MySQL database, login roles, CSV/PDF report export, attendance management and charts for performance analysis.

15. References

Python documentation: https://docs.python.org/3/ ; Git documentation: https://git-scm.com/doc